

テキスト解析 Web API ガイドブック

手を動かして学ぶ 日本語テキスト処理



日本語形態素解析

固有表現抽出

キーフレーズ抽出

ハンズオンで
手軽に体験！

LINEヤフー

テキスト解析 Web API ガイドブック

手を動かして学ぶ 日本語テキスト処理

LINEヤフー株式会社 テキスト解析ガイドブック制作チーム 著

はじめに

LINE ヤフー株式会社では、自然言語処理を Web API として提供する「テキスト解析 Web API」を開発・公開しています。本書では、この Web API の仕組みや基本的な使い方に加えて、ハンズオン形式の実践例や応用的な活用事例も紹介します。手を動かしながら読み進めることで、機能の体験から導入に至るまでの流れを段階的に理解できるよう構成しています。自然言語処理をサービスに取り入れる際の第一歩として、ぜひ本 Web API をご活用ください。

自然言語は人間にとっては自然なコミュニケーション手段ですが、柔軟で曖昧な表現を多く含むため、コンピュータにとっては扱いづらいデータ形式です。こうした特徴を持つ自然言語を機械的に解析・処理するための技術が、自然言語処理（NLP：Natural Language Processing）です。自然言語処理は、機械翻訳、対話システム、要約、検索、分類など、多くの分野に応用されており、現代の情報社会において欠かせない技術のひとつです。

自然言語処理の基礎技術には、文章を形態素に分割する「形態素解析」、文節の関係を分析する「係り受け解析」、人名や数値を抽出する「固有表現抽出」、特徴的な語句を見つける「キーフレーズ抽出」などがあります。テキスト解析 Web API は、これらの基本的な自然言語処理機能を Web API 経由で提供するサービスです。Web API は、異なるソフトウェア間で機能やデータをやりとりする仕組みであり、HTTP / HTTPS などの Web 技術を用いて実現されています。本 Web API も、JSON を使ったデータ交換に対応し、再利用性やスケーラビリティ、他サービスとの統合性に優れています。

近年の生成 AI 技術の進展により、多くの自然言語処理タスクにおいて大規模言語モデル（LLM）が高い精度を発揮するようになっています。複雑な処理では LLM が有効な選択肢となる一方、出力が非構造的で予測しづらい場合があり、処理コストや応答速度が課題となることもあります。これに対し、テキスト解析 Web API は、あらかじめ定義された明確なタスクに対して、安定した品質と速度、そして明確な構造で結果を返すため、LLM に比べて安価かつ確実に目的の処理を実現できる傾向があります。とくに、文法解析や品詞分解などの決定的な処理には適しています。

本書は、テキスト解析 Web API の基本から実践までを段階的に学べる構成です。利用方法、リクエストの書き方、レスポンスデータの構造、開発時の注意点などを順に解説します。テキスト解析 Web API の導入を検討する開発者や、自然言語処理を業務に取り入れたい方に向けて、実践的なガイドとなることを目指しています。

本書の対象読者は、次のような方々です。豊富なサンプルコードを掲載しているため、プログラミングの経験が浅い方でも手を動かしながら自然言語処理を体験できます。

- ・生成 AI の広がり背景に、自然言語処理の初歩から学びたい IT・ウェブ分野のエンジニア
- ・大学や独学で自然言語処理を学び、実務における応用方法を知りたい学生・若手エンジニア
- ・日本語のテキストデータを扱う業務に携わっており、テキストマイニングをこれから始めたい初心者
- ・機械学習を業務に導入しており、日本語テキストを特徴量として活用したいと考えている方

本書は次の 5 章で構成されています。基本的な概念の理解から活用まで段階的にステップアップできる構成です。

- ・ **第 1 章「テキスト解析 Web API の概要」**：本 API の機能概要や特徴、利用形式について概説します。
- ・ **第 2 章「テキスト解析 Web API の利用準備」**：利用に必要なアカウントやツールの準備方法、API キーの取得方法など、初期設定の手順を解説します。
- ・ **第 3 章「テキスト解析 Web API ハンズオン」**：各機能の基本的な使い方を、サンプルコードとともに実践形式で紹介します。本書の中心となる章です。
- ・ **第 4 章「テキスト解析 Web API ハック例」**：ハンズオンの内容を発展させた応用的な活用例を取り上げ、API の実用性や拡張性を紹介します。
- ・ **第 5 章「まとめ」**：本書全体の内容を振り返り、テキスト解析 Web API の要点を整理します。

ご案内

GitHubリポジトリ

テキスト解析 Web API に関連するサンプルコードやドキュメントは、「テキスト解析 Web API クックブック」として GitHub で公開しています。

テキスト解析 Web API クックブック

<https://github.com/yahoojapan/textwebapi-cookbook>



また、モバイル向けのオープンソース SDK「LY Corporation Mobile NLP SDK」を GitHub で公開しています。この SDK を利用することで、Android や iOS アプリからテキスト解析 Web API を容易に利用できます。

LY Corporation Mobile NLP SDK

<https://github.com/yahoojapan/ly-corporation-mobile-nlp-sdk>



動作環境

本書のハンズオンでは、curl と jq を使用したコマンドライン操作を前提としています（Windows / macOS / Linux いずれも対応可能）。詳細な使い方は後述します。

また、第 4 章のハック例では以下の環境で動作確認済みのプログラム例を掲載しています：

- Python：JupyterLab, Jupyter Notebook, Google Colaboratory
- JavaScript + HTML：Google Chrome などのモダンブラウザ

利用規約・ガイドライン

テキスト解析 Web API を利用するにあたっては、以下のガイドライン・規約を事前にご確認ください。

- Yahoo! デベロッパーネットワーク ガイドライン <https://developer.yahoo.co.jp/guideline/>
- 利用制限について <https://developer.yahoo.co.jp/appendix/rate.html>
- 商用利用について <https://support.yahoo-net.jp/PccDeveloper/s/article/H000011080>

情報の更新日

本書に記載された情報は、発行時点の内容に基づいています。今後の技術的な更新や仕様変更に対応するため、最新情報は GitHub リポジトリまたは公式ドキュメントをご参照ください。

本書で使用するコード・リンク

本書で紹介するコマンド例や URL のリンク一覧は、以下の GitHub リポジトリで公開しています。ハンズオンで使用する内容をコピー＆ペーストしてすぐに試せる形でまとめていますので、ぜひご活用ください。

<https://github.com/yahoojapan/textwebapi-cookbook/tree/main/Guidebook>

注記

本書の例文に登場する企業名・人名はすべて架空のものであり、実在のものとは関係ありません。また、地名や施設名は説明上の例であり、特定の企業・団体・施設等との関係を示すものではありません。なお、本書に掲載しているグラフや図は誌面での視認性を高めるためにレイアウトを一部調整しています。

目次

第 1 章 テキスト解析 Web API の概要	1
1.1 テキスト解析 Web API の機能一覧	1
1.2 採用しているインターフェース ～ JSON-RPC 2.0 ～	2
1.3 Web API 形式での提供のメリット	2
1.4 発展：モバイルアプリへの組み込み	3
1.5 まとめ	3
第 2 章 テキスト解析 Web API の利用準備	5
2.1 Client ID の取得	5
2.2 ツールの準備	5
2.3 テキスト解析 Web API の使い方	6
2.4 テキスト解析 Web API 利用の注意点	7
2.5 まとめ	7
第 3 章 テキスト解析 Web API ハンズオン	9
3.1 日本語形態素解析	9
3.1.1 形態素解析とは	9
3.1.2 日本語形態素解析の API 概要	9
3.1.3 実行例	10
3.1.4 ハンズオンの振り返り	11
3.1.5 演習問題	12
3.2 固有表現抽出	12
3.2.1 固有表現抽出とは	12
3.2.2 固有表現抽出の API 概要	12
3.2.3 実行例	13
3.2.4 ハンズオンの振り返り	15
3.2.5 演習問題	15
3.3 キーフレーズ抽出	15
3.3.1 キーフレーズ抽出とは	15
3.3.2 キーフレーズ抽出の API 概要	16
3.3.3 実行例	16
3.3.4 ハンズオンの振り返り	17
3.3.5 演習問題	17
3.4 まとめ	18

第4章 テキスト解析 Web API ハック例

19

4.1	日本語形態素解析 API を用いたハック例	19
4.1.1	形態素や品詞の頻度を集計してグラフ化する	19
4.1.2	本ハック例のまとめ	22
4.2	固有表現抽出 API を用いたハック例	22
4.2.1	テキスト中の固有表現に自動でハイパーリンクを貼る	22
4.2.2	本ハック例のまとめ	24
4.3	キーフレーズ抽出 API を用いたハック例	24
4.3.1	キーフレーズを抽出してワードクラウドを作る	24
4.3.2	本ハック例のまとめ	26
4.4	本章のまとめ	26

第5章 まとめ

27

付録

29

演習問題略解	29
形態素解析	29
固有表現抽出	29
キーフレーズ抽出	30
Web API 仕様概略	31
A-1. 日本語形態素解析	31
A-2. 固有表現抽出	32
A-3. キーフレーズ抽出	33

第1章 テキスト解析 Web API の概要

テキスト解析 Web API は、自然言語処理の基本機能を Web API 経由で簡単に利用できるサービスです。ユーザは、テキストを API に送信するだけで、構造化された解析結果を受け取ることができます。LINE ヤフーでは、このテキスト解析 Web API を Yahoo! デベロッパーネットワークを通じて公開しており、さまざまな言語処理機能を提供しています。

本章では、テキスト解析 Web API で提供している機能の一覧や特徴について説明します。

1.1 テキスト解析 Web API の機能一覧

本書執筆時点では、テキスト解析 Web API は次の 8 つの機能を提供しています。

- ・「日本語形態素解析」- 文を形態素に分割し、品詞などの情報を付与
- ・「かな漢字変換」- ローマ字・ひらがなの文を漢字交じり文に変換
- ・「ルビ振り」- 漢字に読み仮名を付与
- ・「校正支援」- 誤用や誤字脱字などをチェック
- ・「日本語係り受け解析」- 文節間の依存関係を分析
- ・「キーフレーズ抽出」- 文の特徴的なフレーズを抽出
- ・「自然言語理解」- 文の意図やエンティティなどの意味情報を抽出
- ・「固有表現抽出」- 人名・地名・数値などの固有表現を抽出

各機能がどのような処理を行うのかをイメージしやすいよう、代表的な入出力例を以下に示します。

テキスト解析 Web API 各機能の入出力イメージ

機能名	入出力イメージ
日本語形態素解析	入力：明日の天気
	出力：/ 明日 (名詞 あす) / の (助詞 の) / 天気 (名詞 てんき) /
かな漢字変換	入力：きょうはいいてんきです
	出力：今日は / いい / 天気です
ルビ振り	入力：薔薇は美しい
	出力：ばら / は / うつくしい, bara/ ha/ utukusii
係り受け解析	入力：私は黒い猫を飼っている
	出力：私 - 飼っている, 黒い - 猫を
校正支援	入力：セキュリティー, 食べれる
	出力：セキュリティ (用字), 食べられる (ら抜き)
キーフレーズ抽出	入力：東京ミッドタウンから国立新美術館まで
	出力：東京ミッドタウン (100), 国立新美術館 (73)
固有表現抽出	入力：東京で佐藤さんに会う
	出力：東京 (地名), 佐藤 (人名)
自然言語理解	入力：明日の東京の天気を教えて
	出力：意図→天気情報, エンティティ→ 明日 (日付) / 東京 (場所)

これらの処理は、すべて Web API を通じて同じ手順で利用できます。例えば日本語形態素解析では、解析したいテキストを JSON 形式で API に送信すると、構造化された解析結果が返されます（次ページのサンプル参照）。この仕組みは全機能で共通しており、一度使い方を理解すれば、他の処理も同様に利用できます。詳細なフォーマットや仕様は次節で説明します。

リクエスト

```
{
  "id": "1234-1",
  "jsonrpc": "2.0",
  "method": "jlp.maservice.parse",
  "params": {
    "q": "明日の天気"
  }
}
```

レスポンス

```
{
  "id": "1234-1",
  "jsonrpc": "2.0",
  "result": {
    "tokens": [
      ["明日", "あす", "明日", "名詞", "時相名詞", "*", "*"],
      ["の", "の", "の", "助詞", "接続助詞", "*", "*"],
      ["天気", "てんき", "天気", "名詞", "普通名詞", "*", "*"]
    ]
  }
}
```

日本語形態素解析のリクエストとレスポンスのサンプル

1.2 採用しているインターフェース ～ JSON-RPC 2.0 ～

テキスト解析 Web API では、入出力のインターフェースとして JSON-RPC 2.0 (<https://www.jsonrpc.org/specification>) を採用しています。この仕様に基づくフォーマットは、日本語形態素解析をはじめ、すべての機能で共通しており、データの送受信は JSON 形式で行われます。

従来の多くの Web API では、複数の機能を提供していても、入出力形式や設定が機能ごとに異なり、扱いづらい場合があります。一方、テキスト解析 Web API は JSON-RPC 2.0 によるシンプルで統一された設計を採用しており、機能名 (method) と JSON 構造を把握するだけで簡単に各機能呼び出すことができます。

一度この形式に慣れれば、日本語形態素解析はもちろん、キーフレーズ抽出、ルビ振り、校正支援などの他の機能も同様の手順で利用できます。

リクエスト

```
{
  "id": "<リクエストごとのID>",
  "jsonrpc": "2.0",
  "method": "<利用する機能>",
  "params": {
    "q": "<解析したいテキスト>"
  }
}
```

レスポンス

```
{
  "id": "<リクエスト時に指定したID>",
  "jsonrpc": "2.0",
  "result": {
    <解析結果>
  }
}
```

テキスト解析 Web API のリクエスト/レスポンスの形式 (JSON-RPC 2.0)

1.3 Web API 形式での提供のメリット

テキスト解析 Web API は、その名のとおり Web API 形式で提供されています。この方式で言語処理機能を提供することには、さまざまな利点があります。主な利点を以下に示します。

- ・インストールなしで使える

一般的な自然言語処理ツールでは利用者側の環境へのインストールや設定が必要ですが、テキスト解析 Web API はそのような準備が不要です。解析処理はすべてサーバー側で行われるため、利用者は計算資源を意識せずに利用できます。

- ・特別な対応をしなくても最新版を利用できる

モデルの更新やメンテナンスは Web API 側で行われるため、利用者は特別な対応をしなくても最新版を利用できます。更新の手間がなく、安定した環境で利用できます。

- ・他のサービスやアプリに柔軟に組み込める

Web API 形式で提供されており、他のアプリケーションやサービスに柔軟に組み込めます。ネットワーク接続さえあれば利用するため、モバイルアプリなどからも利用可能です。また利用者側のプラットフォームやプログラミング言語を問わず利用できます。

これらの利点により、利用者は手軽に、そして手間をかけずに言語処理機能を利用できます。

1.4 発展：モバイルアプリへの組み込み

Web API 形式の利点を活かし、Android や iOS アプリでもテキスト解析 Web API を簡単に利用できるよう、モバイル向けのオープンソース SDK を公開しています。この SDK をアプリに導入するだけで、複雑な実装や通信処理を意識せずに、日本語形態素解析、ルビ振り、キーフレーズ抽出などの自然言語処理機能をすぐに利用できます。

SDK は、Android 向けに .aar、iOS 向けに .xcframework で提供されており、提供形式は異なるものの、両 OS で共通の仕組みと機能を利用できます。さらに導入手順を確認できるデモアプリも同梱しており、Android Studio や Xcode などの開発環境があれば、すぐに動作を確認できます。

SDK は依存関係が少なく、アプリへの導入が容易になるように設計されています。ただし、Gradle 設定など、各プラットフォーム固有のビルド設定については別途考慮が必要です。また、サーバ側では日本語形態素解析などの自然言語処理機能を継続的に更新しています。そのため、SDK を通じて呼び出す場合も、利用者側で更新作業を行うことなく、その時点で提供されている機能を利用できます。

詳細な導入手順やソースコードは GitHub リポジトリで公開されています。

- GitHub リポジトリ <https://github.com/yahoojapan/ly-corporation-mobile-nlp-sdk>

1.5 まとめ

本章では、テキスト解析 Web API の機能一覧、インターフェース仕様、および Web API 形式の利点について紹介しました。テキスト解析 Web API は、JSON-RPC 2.0 で統一されたインターフェースにより、すべての機能を共通の手順で呼び出せる点が特長です。さらに、Web API 形式で提供されているため、利用者側でインストールや更新作業を行わずに解析処理を実行できます。こうした利便性と柔軟性を背景に、LINE ヤフー社内でも同様のシステムが活用されています。また、モバイル向けのオープンソース SDK を利用すれば、Android や iOS アプリへの組み込みも容易です。次章では、実際にテキスト解析 Web API を利用するための準備手順について詳しく解説します。

1
2
3
4
5

第2章 テキスト解析 Web API の利用準備

本章では、テキスト解析 Web API を利用するために必要な準備について解説します。具体的には、Yahoo! JAPAN が提供する Client ID (アプリケーション ID) の取得方法と、コマンドラインツール (curl および jq) の導入手順を紹介します。これらは、次章で実施するハンズオンをスムーズに進めるための前提となる環境構築です。あわせて、Web API リクエストの基本構造や利用時の注意点・制限事項についても確認します。

2.1 Client ID の取得

テキスト解析 Web API を含む Yahoo! JAPAN の Web サービス (Web API) を使うには Client ID が必要です。Client ID はアプリケーションを特定する文字列であり、すべてのリクエスト送信時に指定します。なお、Client ID はアプリケーション ID (AppID) と呼ばれることもあります。本書では以降 Client ID と表記します。Yahoo! JAPAN ID をお持ちであれば、どなたでも以下の手順で発行できます。

1. <https://e.developer.yahoo.co.jp/register> にアクセス
2. Yahoo! JAPAN ID でログイン (未ログインの場合、ログイン画面に移動)
3. 「新しいアプリケーションを開発」の画面で必要項目を入力
4. Client ID が発行される

本書のサンプルコードでは Client ID が必要な部分に 'あなたの Client ID(アプリケーション ID)' と記載されていますので、取得した ID に置き換えて使用してください。Client ID についての詳細は以下をご覧ください。

<https://support.yahoo-net.jp/PccDeveloper/s/article/H000006122>

2.2 ツールの準備

ハンズオンでは curl と jq のコマンドラインツールを利用します。curl は HTTP リクエストを送信するためのコマンドラインツールで、Web API との通信に使用します。jq は JSON 形式のデータを整形・抽出するためのツールです。

それぞれのツールを環境に応じてインストールしてください。なお、macOS には curl がデフォルトでインストールされています。一方、jq については、Homebrew を使ってインストールすることが可能です。以下のコマンドをターミナルで実行して、jq をインストールしてください。

```
$ brew install jq
```

curl を使用して日本語形態素解析の Web API にリクエストを送信するサンプルです。

```
$ curl -s \  
-X POST \  
-H "Content-Type: application/json" \  
-H "User-Agent: Yahoo AppID: 'あなたの Client ID(アプリケーション ID)'" \  
https://jlp.yahooapis.jp/MAService/V2/parse \  
-d '{  
  "id": "1234-1",  
  "jsonrpc": "2.0",  
  "method": "jlp.maservice.parse",  
  "params": {  
    "q": "美しい水車小屋の娘"  
  }  
' > sample.json
```

-x オプションで POST メソッドを指定します。JSON 形式のリクエストパラメータは -d オプションで指定します。クオート内では改行が含まれていても問題ありません。ダウンロードの進捗やエラー出力を表示させないように -s オプションを指定します。

利用する機能によって、リクエスト送信先のエンドポイント (https://jlp.yahooapis.jp/...), JSON 形式のリクエストパラメータを変更します。一方、-H オプションで指定するリクエストヘッダーはすべての機能で共通です。これらは 2.3 節「テキスト解析 Web API の使い方」で説明します。

jq を使うことで JSON から必要な部分の取り出しや整形をして読みやすくなります。先ほどのリクエストで得られたレスポンスで説明します。

```
$ cat sample.json
```

```
{
  "id": "1234-1",
  "jsonrpc": "2.0",
  "result": {
    "tokens": [
      ["美しい", "うつくしい", "美しい", "形容詞", "*", "イ形容詞イ段", "基本形"],
      ["水車", "すいしゃ", "水車", "名詞", "普通名詞", "*", "*"],
      ["小屋", "こや", "小屋", "名詞", "普通名詞", "*", "*"],
      ["の", "の", "の", "助詞", "接続助詞", "*", "*"],
      ["娘", "むすめ", "娘", "名詞", "普通名詞", "*", "*"]
    ]
  }
}
```

jq で形態素の表記のみを取り出す例です。

```
$ cat sample.json | jq '.result.tokens[] | .[0]'
```

```
"美しい"
"水車"
"小屋"
"の"
"娘"
```

2.3 テキスト解析 Web API の使い方

テキスト解析 Web API は複数の言語処理機能を提供していますが、そのリクエスト方法は共通となっています。例えば、本書では「日本語形態素解析」「キーフレーズ抽出」「固有表現抽出」の機能をハンズオンで体験しますが、いずれも同じ形式でリクエストできます。解析する際は、解析したいテキストを含むデータを Web API の URL に送ります。

ここでは 2.2 節同様に、日本語形態素解析を例に説明します。日本語形態素解析の API にリクエストする例では、以下のリクエスト URL、ヘッダー、リクエストパラメータを設定しています。

```
リクエストURL
https://jlp.yahooapis.jp/MAService/V2/parse

リクエストヘッダー
Content-Type: application/json
User-Agent: Yahoo AppID: 'あなたの Client ID(アプリケーション ID)'
```

```
リクエストパラメータ
{
  "id": "1234",
  "jsonrpc": "2.0",
  "method": "jlp.maservice.parse",
  "params": { "q": "美しい水車小屋の娘" }
}
```

リクエスト URL は機能ごとに異なりますが、リクエストヘッダーは全機能で共通です。Content-Type には application/json を指定し、JSON 形式のデータを送信することを明示します。また、User-Agent には Client ID を記述します。実際の形式は次のとおりです。

```
"User-Agent: Yahoo AppID: gUxspU.....6sG.....Alc-"
```

リクエストパラメータの構造は JSON-RPC 2.0 仕様（1.2 節で言及）に準拠しており、以下のフィールドで構成されます。

- id：任意の数値または文字列
- jsonrpc：常に 2.0 に固定
- method：利用する機能を指定
- params：method に渡す引数を記述するフィールド。利用する機能によって異なるが、共通して q で解析対象のテキストを指定する

レスポンスも JSON-RPC 2.0 に準拠しており、結果は result フィールドに格納されます。ただし、この result の中は機能ごとに異なる仕様となっています。

なお、リクエスト URL、リクエストパラメータの params、レスポンスフィールドの result についてはハンズオンの章で詳しく説明します。

2.4 テキスト解析 Web API 利用の注意点

テキスト解析 Web API の利用において、注意が必要な点がまとめた参考ページを案内します。

テキスト解析 Web API の利用においては、以下の LINE ヤフー共通利用規約 第 2 章「ソフトウェアに関する規則（ガイドライン）」および「Yahoo! デベロッパーネットワーク ガイドライン」の遵守をお願いします。

- ソフトウェアに関する規則（ガイドライン）<https://www.lycorp.co.jp/ja/company/terms/#anc2>
- Yahoo! デベロッパーネットワーク ガイドライン <https://developer.yahoo.co.jp/guideline/>

テキスト解析 Web API を利用するサイトやアプリケーションには、クレジットを表示する必要があります。クレジットの表示方法、配置ルールなどの詳細は以下をご覧ください。

<https://developer.yahoo.co.jp/attribution/>

テキスト解析 Web API には利用回数に制限があります。1 つの Client ID に対し、1 分で 300 回を超えた場合に制限がかかります。制限がかかっている間は「429 Too Many Requests」のエラーレスポンスが返却されます。有償による利用回数の制限変更などには対応できません。利用回数が制限を超えないよう、リクエスト間隔をあけてご利用ください。

利用回数の制限を含む、よくある質問（FAQ）が以下にまとまっていますので、こちらも参考にしてください。

<https://developer.yahoo.co.jp/webapi/jlp/faq.html>

2.5 まとめ

本章では、テキスト解析 Web API の利用方法と、次の章でのハンズオンに必要な環境のセットアップについて説明しました。具体的な内容としては、Client ID の取得方法、コマンドラインツールの curl と jq の利用方法、テキスト解析 Web API へのリクエスト方法を説明しました。あわせて、利用時の注意点や制限事項についても関連ページを紹介しました。次章では、具体的な自然言語処理の機能を用いたハンズオンを通じて、Web API の使い方を実践的に学んでいきます。

1

2

3

4

5

第3章 テキスト解析 Web API ハンズオン

準備が整ったら、いよいよテキスト解析 Web API を実際に試してみましょう。本章では、形態素解析・固有表現抽出・キーフレーズ抽出の3つの機能について、順を追ってハンズオン形式で体験します。それぞれのステップを通じて、Web API の使い方とレスポンスデータの扱い方を実践的に学びましょう。

3.1 日本語形態素解析

3.1.1 形態素解析とは

形態素解析は、文を形態素に分割し、品詞の推定や活用処理、読みを付与する処理です。形態素は、これ以上分解すると意味を持たない、言語学における最小の意味単位です。

形態素解析の例を示します。下記はテキスト解析 Web API で「美しい水車小屋の娘」という文を解析した結果です。

表記	読み	基本形表記	品詞	品詞細分類	活用型	活用形
美しい	うつくしい	美しい	形容詞	*	イ形容詞イ段	基本形
水車	すいしゃ	水車	名詞	普通名詞	*	*
小屋	こや	小屋	名詞	普通名詞	*	*
の	の	の	助詞	接続助詞	*	*
娘	むすめ	娘	名詞	普通名詞	*	*

形態素解析を使うと、例えば「形態素や品詞の頻度を集計してグラフ化する」「キーワードマッチの誤マッチを減らす」「マルコフ連鎖による文生成」といったこともできるようになります。活用例の詳細は第4章や「テキスト解析 Web API クックブック」にも掲載していますので、ぜひご覧ください。

3.1.2 日本語形態素解析の API 概要

リクエストURL

`https://jlp.yahooapis.jp/MAService/V2/parse`

サンプルリクエスト

method には `jlp.maservice.parse` を、params.q には解析したいテキストを指定します。

```
{
  "id": "1234-1",
  "jsonrpc": "2.0",
  "method": "jlp.maservice.parse",
  "params": {
    "q": "美しい水車小屋の娘"
  }
}
```

サンプルレスポンス

`result.tokens[]` に全形態素の情報が出力されます。各形態素の情報は、表記、読み、基本形表記、品詞、品詞細分類、活用型、活用形の順に出力されます。

```
{
  "id": "1234-1",
  "jsonrpc": "2.0",
  "result": {
    "tokens": [
      ["美しい", "うつくしい", "美しい", "形容詞", "*", "イ形容詞イ段", "基本形"],
      ["水車", "すいしゃ", "水車", "名詞", "普通名詞", "*", "*"],
      ["小屋", "こや", "小屋", "名詞", "普通名詞", "*", "*"],
      ["の", "の", "の", "助詞", "接続助詞", "*", "*"],
      ["娘", "むすめ", "娘", "名詞", "普通名詞", "*", "*"]
    ]
  }
}
```

1

2

3

4

5

仕様の詳細につきましては、付録 A-1 および下記の Web ページをご参照ください。

日本語形態素解析 (V2) - Yahoo! デベロッパーズネットワーク: <https://developer.yahoo.co.jp/webapi/jlp/ma/v2/parse.html>

3.1.3 実行例

curl と jq を使った実行例です。jq を使って JSON のレスポンスから全形態素の情報を抽出しています。

```
$ curl -s \
-X POST \
-H "Content-Type: application/json" \
-H "User-Agent: Yahoo AppID: 'あなたの Client ID(アプリケーション ID)'" \
https://jlp.yahooapis.jp/MAService/V2/parse \
-d '{
  "id": "1",
  "jsonrpc": "2.0",
  "method": "jlp.maservice.parse",
  "params": {
    "q": "美しい水車小屋の娘"
  }
}' | jq -c '.result.tokens[]'
```

```
["美しい","うつくしい","美しい","形容詞","*","イ形容詞イ段","基本形"]
["水車","すいしゃ","水車","名詞","普通名詞","*","*"]
["小屋","こや","小屋","名詞","普通名詞","*","*"]
["の","の","の","助詞","接続助詞","*","*"]
["娘","むすめ","娘","名詞","普通名詞","*","*"]
```

分かち書き

他にも jq で出力を加工することでさまざまな用途に使えます。例えば、各形態素の表記だけをスペース区切りで表示することで、いわゆる「分かち書き」ができます。

```
$ curl -s \
-X POST \
-H "Content-Type: application/json" \
-H "User-Agent: Yahoo AppID: 'あなたの Client ID(アプリケーション ID)'" \
https://jlp.yahooapis.jp/MAService/V2/parse \
-d '{
  "id": "1",
  "jsonrpc": "2.0",
  "method": "jlp.maservice.parse",
  "params": {
    "q": "美しい水車小屋の娘"
  }
}' | jq -r '[".result.tokens[] | .[0] ] | join(" ")'
```


美しい 水車 小屋 の 娘

品詞を得る

名詞、形容詞、動詞といった品詞だけを取り出して集計することもできます。

```
$ curl -s \  
-X POST \  
-H "Content-Type: application/json" \  
-H "User-Agent: Yahoo AppID: 'あなたの Client ID(アプリケーション ID)'" \  
https://jlp.yahooapis.jp/MAService/V2/parse \  
-d '{  
  "id": "1",  
  "jsonrpc": "2.0",  
  "method": "jlp.maservice.parse",  
  "params": {  
    "q": "美しい水車小屋の娘"  
  }  
}' | jq -r '[.result.tokens[] | .[3] ] | join(" ")'
```

形容詞 名詞 名詞 助詞 名詞

ユーザ辞書エントリの追加

リクエストにユーザ辞書エントリを含めることで、独自の語彙を反映した形態素解析が可能です。下記の例ではユーザ辞書エントリに「水車小屋」と「東京スカイツリー」を追加しています。これにより、解析結果の「水車小屋」は一語（一形態素）として表示されています。

```
$ curl -s \  
-X POST \  
-H "Content-Type: application/json" \  
-H "User-Agent: Yahoo AppID: 'あなたの Client ID(アプリケーション ID)'" \  
https://jlp.yahooapis.jp/MAService/V2/parse \  
-d '{  
  "id": "1",  
  "jsonrpc": "2.0",  
  "method": "jlp.maservice.parse",  
  "params": {  
    "q": "美しい水車小屋の娘",  
    "context": {  
      "entries": [  
        "水車小屋, すいしゃごや",  
        "東京スカイツリー"  
      ]  
    }  
  }  
}' | jq -c '.result.tokens[]'
```

```
["美しい", "うつくしい", "美しい", "形容詞", "*", "イ形容詞イ段", "基本形"]  
["水車小屋", "すいしゃごや", "水車小屋", "名詞", "固有名詞", "*", "*"]  
["の", "の", "の", "助詞", "接続助詞", "*", "*"]  
["娘", "むすめ", "娘", "名詞", "普通名詞", "*", "*"]
```

3.1.4 ハンズオンの振り返り

- ・形態素解析は、文を形態素に分割し、品詞の推定や活用処理、読みなどの情報を付与する技術
- ・テキスト解析 Web API で日本語形態素解析を利用する時は、method に jlp.maservice.parse を指定する
- ・リクエストにユーザ辞書エントリを追加することもできる

3.1.5 演習問題

ウェブ上の任意のテキストをテキスト解析 Web API で形態素解析してその結果から名詞のみを取り出し、頻度の多い順に表示せよ。

3.2 固有表現抽出

3.2.1 固有表現抽出とは

固有表現抽出は、テキストから人名や地名、組織名、日時、金額などの特定の意味を持つ語句（固有名詞や数値表現）を識別・抽出する技術です。

固有表現抽出の例を示します。下記はテキスト解析 Web API で「X データ株式会社の加藤さんは、10 時に東京駅に到着しました。」という文を解析した結果です。

テキスト	種類	文字オフセット	文字長
Xデータ株式会社	ORGANIZATION	0	8
加藤	PERSON	9	2
10時	TIME	15	3
東京駅	LOCATION	19	3

固有表現抽出を使うと、例えば「テキスト中の固有表現に自動でハイパーリンクを貼る」「テキスト中の固有表現をマスキング」「プロフィールから固有表現を抽出してダイアグラムを作成する」といったこともできるようになります。活用例の詳細は第 4 章や「テキスト解析 Web API クックブック」をご覧ください。

3.2.2 固有表現抽出の API 概要

リクエストURL

```
https://jlp.yahooapis.jp/NERService/V1/extract
```

サンプルリクエスト

method には jlp.nerservice.extract を指定し、params.q に解析したいテキストを指定してください。

```
{
  "id": "1234-1",
  "jsonrpc": "2.0",
  "method": "jlp.nerservice.extract",
  "params": {
    "q": "Xデータ株式会社の加藤さんは、10時に東京駅に到着しました。"
  }
}
```

サンプルレスポンス

result.entityList[] が抽出された固有表現の一覧です。各固有表現のテキスト、種類、文字オフセット、文字長が出力されます。

```
{
  "id": "1234-1",
  "jsonrpc": "2.0",
  "result": {
    "entityList": [
      {
        "length": 8,
        "offset": 0,
        "text": "Xデータ株式会社",
        "type": "ORGANIZATION"
      },
      {
        "length": 2,
        "offset": 9,
        "text": "加藤",
        "type": "PERSON"
      },
      {
        "length": 3,
        "offset": 15,
        "text": "10時",
        "type": "TIME"
      },
      {
        "length": 3,
        "offset": 19,
        "text": "東京駅",
        "type": "LOCATION"
      }
    ]
  }
}
```

仕様の詳細につきましては、付録 A-2 および下記の Web ページをご参照ください。

固有表現抽出 (V1) - Yahoo! デベロッパーネットワーク : <https://developer.yahoo.co.jp/webapi/jlp/ner/v1/index.html>

3.2.3 実行例

curl と jq を使った実行例です。jq を使って JSON のレスポンスから固有表現の一覧を抽出しています。

```
$ curl -s \
-X POST \
-H "Content-Type: application/json" \
-H "User-Agent: Yahoo AppID: 'あなたの Client ID(アプリケーション ID)'" \
https://jlp.yahooapis.jp/NERService/V1/extract \
-d '{
  "id": "1",
  "jsonrpc": "2.0",
  "method": "jlp.nerservice.extract",
  "params": {
    "q": "Xデータ株式会社の加藤さんは、10時に東京駅に到着しました。"
  }
}' | jq -c '.result.entityList[]'
```

```
{ "length": 8, "offset": 0, "text": "Xデータ株式会社", "type": "ORGANIZATION" }
{ "length": 2, "offset": 9, "text": "加藤", "type": "PERSON" }
{ "length": 3, "offset": 15, "text": "10時", "type": "TIME" }
{ "length": 3, "offset": 19, "text": "東京駅", "type": "LOCATION" }
```

LOCATION (地名) だけを取得する

他にも jq で出力を加工することでさまざまな用途に使えます。例えば、種類が LOCATION の固有表現のみを抽出することで、地名の固有表現のみを取得することができます。

```
$ curl -s \
-X POST \
-H "Content-Type: application/json" \
-H "User-Agent: Yahoo AppID: 'あなたの Client ID(アプリケーション ID)'" \
https://jlp.yahooapis.jp/NERService/V1/extract \
-d '{
  "id": "1",
  "jsonrpc": "2.0",
  "method": "jlp.nerservice.extract",
  "params": {
    "q": "Xデータ株式会社の加藤さんは、10時に東京駅に到着しました。"
  }
}' | jq -c '.result.entityList[] | select(.type == "LOCATION")'
```

```
{"length":3,"offset":19,"text":"東京駅","type":"LOCATION"}
```

固有表現の種類とテキストだけを抜き出す

以下のように、種類とテキストだけを抜き出して表示することもできます。

```
$ curl -s \
-X POST \
-H "Content-Type: application/json" \
-H "User-Agent: Yahoo AppID: 'あなたの Client ID(アプリケーション ID)'" \
https://jlp.yahooapis.jp/NERService/V1/extract \
-d '{
  "id": "1",
  "jsonrpc": "2.0",
  "method": "jlp.nerservice.extract",
  "params": {
    "q": "12月25日、星宝株式会社の加藤さんは、新製品「DotAstroPhone」のプロモーションのため、8時に名古屋駅を
        出発し、12時に東京駅に到着しました。その日は30000円以上の購入で10%オフになる特別なキャンペーンを実施
        しました。"
  }
}' | jq -r '.result.entityList[] | [.type, .text] | join(" ")'
```

```
DATE 12月25日
ORGANIZATION 星宝株式会社
PERSON 加藤
ARTIFACT DotAstroPhone
TIME 8時
LOCATION 名古屋駅
TIME 12時
LOCATION 東京駅
MONEY 30000円
PERCENT 10%
```

固有表現の種類別に出現回数をカウントする

種類別に出現回数をカウントすることで、例えばテキスト中にどのような種類の固有表現が多く含まれているのかを確認できます。

```
$ curl -s \
-X POST \
-H "Content-Type: application/json" \
-H "User-Agent: Yahoo AppID: 'あなたの Client ID(アプリケーション ID)'" \
https://jlp.yahooapis.jp/NERService/V1/extract \
-d '{
  "id": "1",
  "jsonrpc": "2.0",
  "method": "jlp.nerservice.extract",
  "params": {
    "q": "12月25日、星宝株式会社の加藤さんは、新製品「DotAstroPhone」のプロモーションのため、8時に名古屋駅を
    出発し、12時に東京駅に到着しました。その日は30000円以上の購入で10%オフになる特別なキャンペーンを実施
    しました。"
  }
}' | jq -r ' [.result.entityList[] | group_by(.type) | map({type: .[0].type, count: length})
|.[] | [.type, .count] | join(" ")'
```

```
ARTIFACT 1
DATE 1
LOCATION 2
MONEY 1
ORGANIZATION 1
PERCENT 1
PERSON 1
TIME 2
```

3.2.4 ハンズオンの振り返り

- ・固有表現抽出は、テキストから固有名詞や数値表現を抽出する技術
- ・テキスト解析 Web API で固有表現抽出を利用する時は、method に jlp.nerservice.extract を指定する
- ・結果を加工することで、特定の固有表現のみの抽出や、固有表現の種類ごとの集計などが容易に行える

3.2.5 演習問題

ウェブ上の任意のテキストをテキスト解析 Web API で固有表現抽出してその結果から地名のみを取り出し、Yahoo! マップのキーワード検索用 URL を作成せよ。

Yahoo! マップの URL の形式は「https://map.yahoo.co.jp/search?q=<クエリ>」です。

3.3 キーフレーズ抽出

3.3.1 キーフレーズ抽出とは

キーフレーズ抽出は、入力された文章から、内容を表す特徴的な語句（キーフレーズ）を自動的に抽出する技術です。キーフレーズは、「時計」のように一単語からなるものや、「誕生日プレゼント」のように連続する複数の単語からなるものがあります。

以下は、「東京ミッドタウンから国立新美術館まで歩いて 5 分で着きます。」という文をテキスト解析 Web API に入力した際のキーフレーズ抽出結果の一例です。

スコア	テキスト
100	東京ミッドタウン
73	国立新美術館
37	5分

テキスト解析 Web API では、抽出されたキーフレーズにスコアが付与されています。スコアはキーフレーズの重要度を示しており、数値が大きいほどその語句が文中で特徴的であると判断されています。キーフレーズ抽出を使うと、「キーフレーズ

ズを抽出してワードクラウドを作る」「文中のキーフレーズに色を付けて視覚的に表示する」「キーフレーズを用いて長文の類似度を判定する」といったこともできます。活用例の詳細は第4章や「テキスト解析 Web API クックブック」をご覧ください。

3.3.2 キーフレーズ抽出の API 概要

リクエストURL

```
https://jlp.yahooapis.jp/KeyphraseService/V2/extract
```

サンプルリクエスト

method には jlp.keyphraseservice.extract を指定し、params.q に解析したいテキストを指定してください。

```
{
  "id": "1234-1",
  "jsonrpc": "2.0",
  "method": "jlp.keyphraseservice.extract",
  "params": {
    "q": "東京ミッドタウンから国立新美術館まで歩いて5分で着きます。"
  }
}
```

サンプルレスポンス

result.phrases[] が抽出されたキーフレーズの一覧です。各キーフレーズのスコアとテキストが出力されます。

```
{
  "id": "1234-1",
  "jsonrpc": "2.0",
  "result": {
    "phrases": [
      {
        "score": 100,
        "text": "東京ミッドタウン"
      },
      {
        "score": 73,
        "text": "国立新美術館"
      },
      {
        "score": 37,
        "text": "5分"
      }
    ]
  }
}
```

仕様の詳細につきましては、付録 A-3 および下記の Web ページをご参照ください。

キーフレーズ抽出 (V2) - Yahoo! デベロッパーネットワーク:

<https://developer.yahoo.co.jp/webapi/jlp/keyphrase/v2/extract.html>

3.3.3 実行例

curl と jq を使った実行例です。jq を使って JSON のレスポンスからキーフレーズの一覧を抽出しています。

```
$ curl -s \
-X POST \
-H "Content-Type: application/json" \
-H "User-Agent: Yahoo AppID: 'あなたの Client ID(アプリケーション ID)'" \
https://jlp.yahooapis.jp/KeyphraseService/V2/extract \
-d '{
  "id": "1",
  "jsonrpc": "2.0",
  "method": "jlp.keyphraseservice.extract",
  "params": {
    "q": "東京ミッドタウンから国立新美術館まで歩いて5分で着きます。"
  }
}' | jq -c '.result.phrases[]'
```

```
{ "score": 100, "text": "東京ミッドタウン" }
{ "score": 73, "text": "国立新美術館" }
{ "score": 37, "text": "5分" }
```

スコア上位 5 件のキーフレーズを取得する

キーフレーズ抽出では、スコアの高い順に最大 20 件まで結果が返されます。必要に応じて、その中から上位 5 件のみを表示することも可能です。

```
$ curl -s \
-X POST \
-H "Content-Type: application/json" \
-H "User-Agent: Yahoo AppID: 'あなたの Client ID(アプリケーション ID)'" \
https://jlp.yahooapis.jp/KeyphraseService/V2/extract \
-d '{
  "id": "1",
  "jsonrpc": "2.0",
  "method": "jlp.keyphraseservice.extract",
  "params": {
    "q": "誕生日プレゼントに時計をもらいました。スマートウォッチで、とても多彩な機能があり、中でも健康管理が気に入ります。防水仕様なのでちょっとしたアウトドアスポーツでも使うことができます。"
  }
}' | jq -r 'limit(5; .result.phrases[].text)'
```

```
アウトドアスポーツ
スマートウォッチ
防水仕様
時計
誕生日プレゼント
```

3.3.4 ハンズオンの振り返り

- ・キーフレーズ抽出は、文から特徴的なフレーズを抽出する技術
- ・テキスト解析 Web API でキーフレーズ抽出を利用する時は、method に jlp.keyphraseservice.extract を指定する
- ・テキスト解析 Web API では、各キーフレーズにスコアが付与された状態で返されるため、重要度に応じた処理や可視化が容易に行える

3.3.5 演習問題

ウェブ上の任意のテキストをテキスト解析 Web API でキーフレーズ抽出してその結果を HTML 形式で出力せよ。その際にキーフレーズのフォントサイズをスコアに準じて設定し、スコアが大きいほど文字が大きくなるようにせよ。

3.4 まとめ

本章では、形態素解析・固有表現抽出・キーフレーズ抽出の3つの基本機能について、ハンズオン形式で使い方と結果の紹介をしました。いずれも、`curl` を用いたリクエスト送信と、`jq` によるレスポンス整形によって、構造化されたデータを簡単に取得・活用できることが確認できました。

また、本章で紹介した各種の解析機能を組み合わせたり加工したりすることで、より実践的で高度なテキスト処理も実現できます。次章では、こうした機能を応用した「ハック例」を紹介し、実務や開発現場での活用アイデアをさらに広げていきます。

1

2

3

4

5

第4章 テキスト解析 Web API ハック例

本章では、テキスト解析 Web API の日本語形態素解析、固有表現抽出、キーフレーズ抽出のハック例を紹介します。GitHub に公開している「テキスト解析 Web API クックブック」には、本章で紹介するハック例に加えて、多岐にわたるテキスト解析 Web API の事例が掲載されていますのでご参照ください。なお、本ハック例で使用しているプログラミング言語は Python, JavaScript (+ HTML) です。

4.1 日本語形態素解析 API を用いたハック例

4.1.1 形態素や品詞の頻度を集計してグラフ化する

テキスト解析 Web API の日本語形態素解析の機能を用いて、形態素の表記と品詞の頻度分布を調査します。結果は円グラフや棒グラフで視覚化します。コード例は GitHub のテキスト解析 Web API クックブックにありますので、参考にしてください。

- GitHub テキスト解析 Web API クックブック「形態素や品詞の頻度を集計してグラフ化する」

https://github.com/yahoojapan/textwebapi-cookbook/blob/main/03_API_Examples/02_MAService_FrequencyCount.md

ソースコードの解説

JupyterLab, Jupyter Notebook, Google Colaboratory で動かす Python プログラムの説明です。実行環境の設定方法は省略しますが、Google アカウントがあれば Google Colaboratory が便利です。

まず、必要なモジュールを import します。

```
import pandas as pd
import json
from urllib import request
import japanize_matplotlib
```

モジュールの import エラーが発生した場合は、pip コマンドを使用して必要なモジュールをインストールしてください。japanize_matplotlib が未インストールの場合は、以下のコードを実行してインストールしてください。

```
!pip install japanize_matplotlib
```

次に、Web API にアクセスして形態素解析を行う関数を定義します。

```

APPID = "あなたの Client ID(アプリケーション ID)" # ←A: Client IDの定義
URL = "https://jlp.yahooapis.jp/MAService/V2/parse"

headers = {
    "Content-Type": "application/json",
    "User-Agent": "Yahoo AppID: {}".format(APPID)
}

param_dic = {
    "id": "1234-1",
    "jsonrpc": "2.0",
    "params": {}
}

def ma(query): # ←B: 日本語形態素解析のAPIにリクエストする関数の定義
    param_dic['params']['q'] = query
    param_dic['method'] = "jlp.maservice.parse"
    params = json.dumps(param_dic).encode()
    req = request.Request(URL, params, headers)
    with request.urlopen(req) as res:
        body = res.read()
    return body.decode()

```

本実装は Yahoo! デベロッパーネットワークのサンプル (<https://developer.yahoo.co.jp/webapi/jlp/sample/sample14.html>) で紹介している Python サンプルプログラムとほぼ同じです。Client ID (← A) については 2.1 節の「Client ID の取得」を参照してください。関数 ma (← B) は、引数として渡されたテキストをテキスト解析 Web API の日本語形態素解析にリクエストし、その解析結果を JSON 形式で取得します。リクエストパラメータの method には jlp.maservice.parse を指定しています。

動作確認

では、実際に動作確認してみましょう。まず、日本語のテキストを形態素解析にかけて、その結果を JSON 文字列として取得し、その JSON 文字列をオブジェクトに変換します。ここでは、例文として日本国憲法の条文の一部を使用します。

```

# 日本語テキストを渡して形態素解析します
text = "国会は、国権の最高機関であって、国の唯一の立法機関である。\\
国会は、衆議院及び参議院の両議院でこれを構成する。\\
両議院は、全国民を代表する選挙された議員でこれを組織する。\\
両議院の議員の定数は、法律でこれを定める。\\
両議院の議員及びその選挙人の資格は、法律でこれを定める。\\
但し、人種、信条、性別、社会的身分、門地、教育、財産又は収入によって差別してはならない。"
response = ma(text)

# 解析結果の JSON 文字列をオブジェクトに変換します
obj = json.loads(response)

```

解析結果の形態素情報を Pandas の DataFrame に保存します。

```

df = pd.DataFrame(
    obj['result']['tokens'],
    columns=['表記', '読み', '基本形表記', '品詞', '品詞細分類', '活用型', '活用形']
)
df.head()

```

最初の 5 行は下記のような結果となります。

表記	読み	基本形表記	品詞	品詞細分類	活用型	活用形
国会	こっかい	国会	名詞	普通名詞	*	*
は	は	は	助詞	副助詞	*	*
、	、	、	特殊	読点	*	*
国権	こっけん	国権	名詞	普通名詞	*	*
の	の	の	助詞	接続助詞	*	*

続いて、出現頻度上位 5 件の品詞とその頻度を確認します。

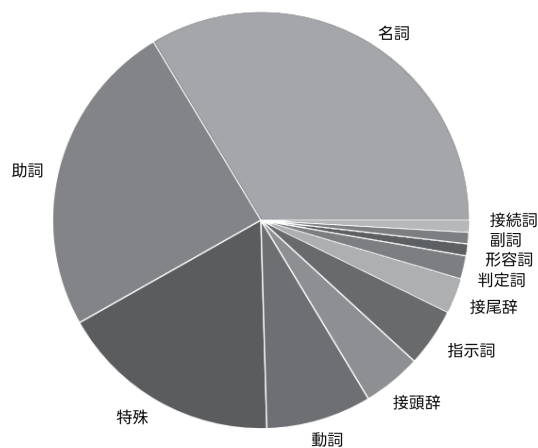
```
df['品詞'].value_counts().head()
```

名詞	37
助詞	27
特殊	19
動詞	9
接頭辞	5

次に、品詞の出現頻度分布を円グラフで表示します。

```
df['品詞'].value_counts().plot.pie(label='')

```



さらに、名詞の形態素を対象に表記の出現回数を集計します。

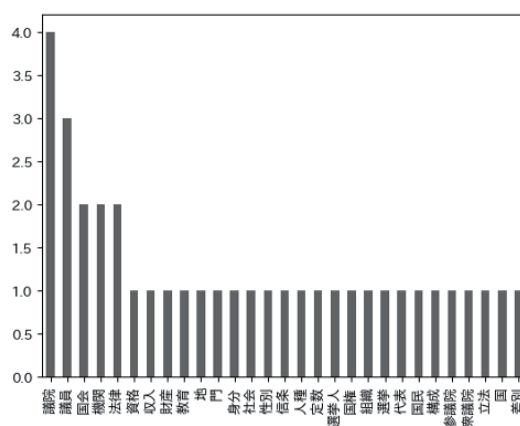
```
df[df['品詞'] == '名詞']['表記'].value_counts().head()
```

ここでは、上位 5 件の名詞とその出現回数を表示しています。

議院	4
議員	3
国会	2
機関	2
法律	2

最後に、名詞の出現頻度分布を棒グラフで表示します。

```
df[df['品詞'] == '名詞']['表記'].value_counts().plot.bar()
```



4.1.2 本ハック例のまとめ

テキスト解析 Web API の日本語形態素解析を使ったハック例を紹介しました。形態素や品詞ごとに集計して分析することで、文章内の重要な表現やテーマを視覚的に理解できます。また、使用頻度の高い単語を特定し、グラフとして提示することで直感的な解釈が可能になります。

4.2 固有表現抽出 API を用いたハック例

4.2.1 テキスト中の固有表現に自動でハイパーリンクを貼る

テキスト解析 Web API の固有表現抽出を利用したハック例です。3.2.5 項のハンズオンの演習問題では、テキストから LOCATION だけを取り出して Yahoo! マップのキーワード検索 URL を作成しましたが、本ハック例はそれを発展させたものです。固有表現が LOCATION のときは Yahoo! マップの URL を、PERSON や ORGANIZATION, ARTIFACT のときは Wikipedia の URL を作って、HTML でリンクを貼っています。コード例は GitHub のテキスト解析 Web API クックブックにありますので、参考してください。

- GitHub テキスト解析 Web API クックブック「テキスト中の固有表現に自動でハイパーリンクを貼る」
https://github.com/yahoojapan/textwebapi-cookbook/blob/main/03_API_Examples/09_NERService_CORSDemo.md

ソースコード解説

固有表現抽出は、ORGANIZATION (組織名)、PERSON (人名)、LOCATION (地名)、ARTIFACT (固有物名)、DATE (日付表現)、TIME (時間表現)、MONEY (金額表現)、PERCENT (割合表現) の 8 種類の固有表現をテキスト中から抽出します。本ハック例では ORGANIZATION (組織名)、PERSON (人名)、LOCATION (地名)、ARTIFACT (固有物名) の 4 種類のみ扱います。本ハック例でのプログラミング言語は HTML と JavaScript です。

次のページのコード全体を参照してください。以下では、重要な処理について順に説明します。

• Client ID の定義 (← A)

ここでは、Web API リクエストに使用する Client ID を定義しています。Client ID については 2.1 節の「Client ID の取得」を参照してください。

• yapiner 関数で解析し、HTML リンクを生成する関数の定義 (← B)

do_henkan 関数は、yapiner 関数の固有表現抽出結果に基づいて処理します。結果がない場合は終了し、ある場合は make_view 関数で HTML リンクを生成します。

• 固有表現抽出の API にリクエストする関数の定義 (← C)

yapiner 関数は、引数の query を固有表現抽出の API にリクエストし解析します。リクエストパラメータには jlp.nerservice.extract を指定し、POST リクエストを送信し、結果を JSON データとして受け取ります。

• HTML リンクを生成する関数の定義 (← D)

固有表現抽出の API から抽出した固有表現に基づいてリンクを生成します。固有表現の種類によって異なるリンク URL を生成し、テキスト内に HTML 形式で埋め込みます。

• HTML 構造の基本部分 (← E)

ユーザがテキストを入力するエリア、解析を実行するボタン、そして解析結果を表示するための出力エリアを指定しています。

```

<!DOCTYPE html>
<html lang="ja">
  <head>
    <meta http-equiv="Content-Type" content="text/html; charset=UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>テキスト解析 Web API 固有表現抽出デモ</title>
    <style>
      * { box-sizing: border-box; }
      textarea, div { width: 100%; margin-top: 1rem; padding: 0.5rem; font-size: 1rem; }
      #view { border: dashed 2px gray; }
    </style>

    <script>
      const ClientID = 'あなたの Client ID(アプリケーション ID)'; // ←A: ClientID の定義

      async function do_henkan() { // ←B: yapiner関数で解析しHTMLリンクを生成する関数の定義
        document.querySelector("#view").innerHTML = '';
        const text = document.querySelector("#input-text").value;
        const obj = await yapiner(text);
        if (! obj?.result?.entityList) return;
        make_view(text, obj);
      }
      async function yapiner(query) { // ←C: 固有表現抽出のAPIにリクエストする関数の定義
        const url = "https://jlp.yahooapis.jp/NERService/V1/extract?appid=" +
          encodeURIComponent(ClientID);
        const res = await fetch(url, {
          method: 'POST',
          mode: 'cors',
          body: JSON.stringify({
            "id": "A123",
            "jsonrpc" : "2.0",
            "method" : "jlp.nerservice.extract",
            "params" : { "q" : query }
          }),
        });
        return res.json();
      }

      function make_view(text, obj) { // ←D: HTMLリンクを生成する関数の定義
        [...obj.result.entityList].reverse().forEach(s => {
          const pre = text.substring(0, parseInt(s.offset));
          const post = text.substring(parseInt(s.offset) + parseInt(s.length));
          if (! /^(ORGANIZATION|PERSON|LOCATION|ARTIFACT)$/.test(s.type)) return;
          const url = (/^LOCATION$/).test(s.type) ?
            'https://map.yahoo.co.jp/search?q=' :
            'https://ja.wikipedia.org/w/index.php?search='
            + encodeURIComponent(s.text);
          text = `${pre}<a target="_blank" href="${url}">${s.text}</a>${post}`;
        });
        document.querySelector("#view").innerHTML = text.replaceAll('\n', '<br>\n');
      }
    </script>
  </head>

  <body> <!-- ←E: HTML構造の基本部分-->
    <h1>テキスト解析 Web API 固有表現抽出デモ</h1>
    <textarea id="input-text" placeholder="入力エリア"></textarea>
    <button onclick="do_henkan()">実行</button>
    <div id="view">出力エリア</div>

    <div style="text-align: right">
      <!-- Begin Yahoo! JAPAN Web Services Attribution Snippet -->
      <span style="margin:15px 15px 15px 15px">
        <a href="https://developer.yahoo.co.jp/sitemap/">Web Services by Yahoo! JAPAN</a></span>
      <!-- End Yahoo! JAPAN Web Services Attribution Snippet -->
    </div>
  </body>
</html>

```

「テキスト中の固有表現に自動でハイパーリンクを貼る」のソースコード

動作確認

では、動作確認をしてみましょう。作成したコードをブラウザで表示させるとデモ画面が表示されます。一番上の「入力エリア」にハイパーリンクを埋め込みたい文章を入力し、その下の「実行」ボタンを押すと、一番下の「出力エリア」にハイパーリンク入りのレイアウト済 HTML テキストが表示されます。それぞれのハイパーリンクは、LOCATION の場合は Yahoo! マップのキーワード検索、ORGANIZATION、PERSON、ARTIFACT の場合は Wikipedia のキーワード検索へのリンクとなっています。



4.2.2 本ハック例のまとめ

テキスト解析 Web API の固有表現抽出を使ったハック例を紹介しました。文中から人名・地名・組織名などを抽出し、それぞれに適したリンクを動的に付与することで、情報へのアクセス性を高める活用例を示しました。このように、抽出結果を外部サービスと連携させることで、テキストの情報を直感的かつ効率的に利用できます。

4.3 キーフレーズ抽出 API を用いたハック例

4.3.1 キーフレーズを抽出してワードクラウドを作る

最後に紹介するのは、テキスト解析 Web API のキーフレーズ抽出です。ここでは、キーフレーズ抽出の結果を用いて、ワードクラウドで可視化します。ワードクラウドとは、テキストに含まれる単語を出現頻度や重要度に応じて文字サイズを変えて表示するもので、情報の重要な部分を直感的に把握することが可能です。本ハック例ではプログラミング言語の Python を使用します。コード例は GitHub のテキスト解析 Web API クックブックにありますので、参考にしてください。

- GitHub テキスト解析 Web API クックブック「キーフレーズを抽出してワードクラウドを作る」

https://github.com/yahoojapan/textwebapi-cookbook/blob/main/03_API_Examples/03_KeyphraseService_Wordcloud.md

ソースコードの解説

JupyterLab, Jupyter Notebook, Google Colaboratory など動かすことを念頭に置いた Python プログラムの流れについて説明します。ここでは実行環境の準備については触れません。Google アカウントがあれば、Google Colaboratory を利用できます。

まず、テキスト解析 Web API のキーフレーズ抽出にリクエストを送信する `keyphrase_service` 関数を定義します。

```

import json
from urllib import request

APPID = "あなたの Client ID(アプリケーション ID)" # ←A: Client IDの定義
URL = "https://jlp.yahooapis.jp/KeyphraseService/V2/extract"
headers = {
    "Content-Type": "application/json",
    "User-Agent": "Yahoo AppID: {}".format(APPID)
}
param_dic = {
    "id": "1234-1",
    "jsonrpc": "2.0",
    "params": {}
}

def keyphrase_service(query): # ←B: キーフレーズ抽出のAPIにリクエストする関数の定義
    param_dic['params']['q'] = query
    param_dic['method'] = "jlp.keyphraseservice.extract"
    params = json.dumps(param_dic).encode()
    req = request.Request(URL, params, headers)
    with request.urlopen(req) as res:
        body = res.read()
    return json.loads(body.decode())['result']['phrases']

```

Client ID の定義(←A)については2.1節の「Client ID の取得」を参照してください。また、keyphrase_service 関数(←B)のリクエストパラメータの method には jlp.keyphraseservice.extract を指定します。

動作確認

それでは動作確認を行しましょう。まず、以下のように日本語の文章を keyphrase_service 関数に渡してください。

```
keyphrase_service('そういえば夏休みに東京タワーに行ったのを思い出しました')
```

以下のように「夏休み」と「東京タワー」が取れたことが確認できます。

```
[{'score': 100, 'text': '東京タワー'}, {'score': 80, 'text': '夏休み'}]
```

続いて、サンプルのテキストをセットします。ここでは、日本国憲法から取ってきた6つの文を使用します。

```

sentences = [
    "国会は、国権の最高機関であって、国の唯一の立法機関である。",
    "国会は、衆議院及び参議院の両議院でこれを構成する。",
    "両議院は、全国民を代表する選挙された議員でこれを組織する。",
    "両議院の議員の定数は、法律でこれを定める。",
    "両議院の議員及びその選挙人の資格は、法律でこれを定める。",
    "但し、人種、信条、性別、社会的身分、門地、教育、財産又は収入によって差別してはならない。",
]

```

次に、各文に対してキーフレーズ抽出を行い、得られた結果からテキスト部分だけを取り出して出現頻度を計算します。

```

# リスト要素ごとにキーフレーズ抽出を行う
kps_list = [keyphrase_service(sentence) for sentence in sentences]

# キーフレーズのテキストのみを取り出す
all_keyphrases = [kp['text'] for kps in kps_list for kp in kps]

# キーフレーズの出現頻度を計算する
import collections
ct_keyphrases = collections.Counter(all_keyphrases)

```

ct_keyphrases には Python の辞書形式で各テキストの頻度が格納されており、例えば次のような内容になります。

{ '兩議院': 4, '議員': 3, '国会': 2, '法律': 2, '国権': 1, ... }

最後に、キーフレーズの頻度情報を用いてワードクラウドを描画します。

```
import matplotlib.pyplot as plt
from wordcloud import WordCloud

wordcloud = WordCloud(
    font_path='C:/Windows/Fonts/HGRSGU.TTC',
    background_color="white", width=600, height=400,
    collocations=False, colormap='Dark2', random_state=0
)

wordcloud.generate_from_frequencies(ct_keyphrases)

fig, ax = plt.subplots(1,1, figsize=(10,10))
ax.axes.xaxis.set_visible(False)
ax.axes.yaxis.set_visible(False)
ax.imshow(wordcloud)
```

font path の設定に関しては以下の表を参考にしてください。

環境	font_path	補足
Google Colaboratory	/usr/share/fonts/opentype/ipafont-gothic/ipagp.ttf	事前に以下のコマンドを実行する <code>!apt-get -y install fonts-ipafont-gothic</code>
Windows	C:/Windows/Fonts/HGRSGU.TTC	
Windows (WSL)	/mnt/c/Windows/Fonts/HGRSGU.TTC	Windows のフォントを WSL から参照
macOS	/Library/Fonts/Arial Unicode.ttf	
Linux	/usr/share/fonts/ipa-gothic/ipag.ttf	fc-list コマンドでパスを確認可能

上記のセルを実行すると、以下のようなワードクラウドの結果が表示されます。



4.3.2 本ハック例のまとめ

テキスト解析 Web API のキーフレーズ抽出を使ったハック例を紹介しました。抽出されたキーフレーズをもとに生成したワーククラウドは、テキスト全体の特徴や傾向を視覚的に把握するための有効な手段となります。

4.4 本章のまとめ

本章では、テキスト解析 Web API の日本語形態素解析、固有表現抽出、キーフレーズ抽出を活用した実践的なハック例を紹介しました。これらの機能を適切に組み合わせることで、大量のテキストから有用な情報を効率的に抽出・可視化することが可能になります。さらに、Python や JavaScript などの汎用的なプログラミング言語と組み合わせることで、Web アプリケーションやデータ分析ツールへの応用も容易になります。

第 5 章 まとめ

本書では、テキスト解析 Web API の基本機能とその活用方法を実行例やハンズオンを通じて段階的に紹介してきました。

第 1 章・第 2 章では、提供されている言語処理機能の概要や、Web API を利用するための準備手順、リクエスト・レスポンスの形式について確認しました。第 3 章では、形態素解析・固有表現抽出・キーフレーズ抽出を実際に動かしながら、使い方とデータの扱い方を学びました。第 4 章では、Web API の解析結果を使った応用例を取り上げ、より実践的な活用イメージを広げました。

本書で扱ったテキスト解析 Web API は、導入が容易で汎用的な開発環境からでもすぐに利用可能な点が大きな魅力です。Web API として提供されているため、ローカル環境に依存せずに安定した解析精度を得られます。また、生成 AI とは異なり、決定的な処理が求められるシーンでも有効に機能します。

今後、自然言語処理の活用を検討している開発者や、既存業務で日本語テキストを扱っている方にとって、テキスト解析 Web API は強力な選択肢となります。本書で紹介した基礎的な使い方や応用例が、皆さまの業務やプロジェクトにおける活用の一助となれば幸いです。

1

2

3

4

5

付録

演習問題略解

形態素解析

ウェブ上の任意のテキストをテキスト解析 Web API で形態素解析してその結果から名詞のみを取り出し、頻度の多い順に表示せよ。

下記のようなコマンドを実行します。params.q には解析したいテキストを指定してください。

```
$ curl -s \  
-X POST \  
-H "Content-Type: application/json" \  
-H "User-Agent: Yahoo AppID: 'あなたの Client ID(アプリケーション ID)'" \  
https://jlp.yahooapis.jp/MAService/V2/parse \  
-d '{  
  "id": "1",  
  "jsonrpc": "2.0",  
  "method": "jlp.maservice.parse",  
  "params": {  
    "q": "日本の切手は..."  
  }  
' | jq -r ' [.result.tokens[] | select(. [3] == "名詞") | . [0] ] | group_by(.)  
  | map ({key: . [0], value: length}) | sort_by(.value) | reverse'
```

```
[  
  {  
    "key": "日本", "value": 8  
  },  
  {  
    "key": "切手", "value": 8  
  },  
  {  
    "key": "発行", "value": 5  
  },  
  ...  
]
```

形態素解析の結果に対して、出現頻度をカウントして降順にソートする箇所がポイントです。手軽に解析結果を集計できる方法であり、さまざまな場面で活用できます。

固有表現抽出

ウェブ上の任意のテキストをテキスト解析 Web API で固有表現抽出してその結果から地名のみを取り出し、Yahoo! マップのキーワード検索用 URL を作成せよ。

下記のようなコマンドを実行します。params.q には解析したいテキストを指定してください。

```
$ curl -s \
-X POST \
-H "Content-Type: application/json" \
-H "User-Agent: Yahoo AppID: 'あなたの Client ID(アプリケーション ID)'" \
https://jlp.yahooapis.jp/NERService/V1/extract \
-d '{
  "id": "1",
  "jsonrpc": "2.0",
  "method": "jlp.nerservice.extract",
  "params": {
    "q": "札幌市時計台(さっぽろしとけいだい)は、..."
  }
}' | jq -r '.result.entityList[] | select(.type == "LOCATION") |
"https://map.yahoo.co.jp/search?q=\(.text)" | sort -u
```

```
https://map.yahoo.co.jp/search?q=さっぽろ
https://map.yahoo.co.jp/search?q=北海道札幌市中央区北1条西2丁目
https://map.yahoo.co.jp/search?q=旧札幌農学校演武場
https://map.yahoo.co.jp/search?q=時計台
https://map.yahoo.co.jp/search?q=札幌市時計台
https://map.yahoo.co.jp/search?q=札幌時計台
```

抽出された固有表現のうち type が LOCATION のもののみを抜き出し、固有表現のテキストを Yahoo! マップのキーワード検索用 URL のパラメータ部分に埋め込んでいます。この解答例では sort コマンドで重複処理を行っています。

キーフレーズ抽出

ウェブ上の任意のテキストをテキスト解析 Web API でキーフレーズ抽出してその結果を HTML 形式で出力せよ。その際にキーフレーズのフォントサイズをスコアに準じて設定し、スコアが大きいほど文字が大きくなるようにせよ。

下記のようなコマンドを実行します。params.q には解析したいテキストを指定してください。

```
$ curl -s \
-X POST \
-H "Content-Type: application/json" \
-H "User-Agent: Yahoo AppID: 'あなたの Client ID(アプリケーション ID)'" \
https://jlp.yahooapis.jp/KeyphraseService/V2/extract \
-d '{
  "id": "1",
  "jsonrpc": "2.0",
  "method": "jlp.keyphraseservice.extract",
  "params": {
    "q": "幕末の..."
  }
}' | jq -rc '.result.phrases[] | "<span style=\"font-size: \(.score)%\">\(.text)</span>" \
> a.html
```

a.html の中身は以下のようになっています。

```
<span style="font-size: 100%">ジャポニズム</span>
<span style="font-size: 84%">開国</span>
<span style="font-size: 73%">西欧</span>
<span style="font-size: 62%">幕末</span>
...
```

この html ファイルを Web ブラウザで開くと、スコアが大きいキーフレーズほど大きな文字で表示されます。

ジャポニズム 開国 西欧 幕末 画家 ロンドン万国博覧会

パリ万国博覧会 浮世絵 アール・ヌーヴォー アーツ・アンド・クラフツ運動

ナビ派 美術運動 芸術作品 印象派 文化 日本文化ブーム 影響 慶応3年 ヨーロッパ中 文久2年 世界 日本

Web API 仕様概略

A-1. 日本語形態素解析

リクエスト URL

https://jlp.yahooapis.jp/MAService/V2/parse

リクエストパラメータ

パラメータ	必須	値	説明
id	○	string, integer	例: 1-234-D
jsonrpc	○	string	2.0 (固定)
method	○	string	jlp.maservice.parse (固定)
params	○	object	
params/q	○	string	解析対象テキスト
params/ context		object	
params/ context/ entries		array (string)	ユーザ辞書として追 加したいエントリ

サンプルリクエスト

```
{
  "id": "1234-1",
  "jsonrpc": "2.0",
  "method": "jlp.maservice.parse",
  "params": {
    "q": "美しい水車小屋の娘"
  }
}
```

レスポンスフィールド

フィールド	値	説明
id	string, integer	リクエストの id の値が返ります
jsonrpc	string	固定で「2.0」が返ります
result	object	形態素解析結果
result/ tokens	array (array)	形態素情報のリスト
result/ tokens/0	string	(形態素情報) 表記
result/ tokens/1	string	(形態素情報) 読み
result/ tokens/2	string	(形態素情報) 基本形表記
result/ tokens/3	string	(形態素情報) 品詞
result/ tokens/4	string	(形態素情報) 品詞細分類
result/ tokens/5	string	(形態素情報) 活用型
result/ tokens/6	string	(形態素情報) 活用形

サンプルレスポンス

```
{
  "id": "1234-1",
  "jsonrpc": "2.0",
  "result": {
    "tokens": [
      ["美しい", "うつくしい", "美しい", "形容詞",
        "*", "イ形容詞イ段", "基本形"],
      ["水車", "すいしゃ", "水車", "名詞",
        "普通名詞", "*", "*"],
      ["小屋", "こや", "小屋", "名詞", "普通名詞",
        "*", "*"],
      ["の", "の", "の", "助詞", "接続助詞", "*",
        "*"],
      ["娘", "むすめ", "娘", "名詞", "普通名詞",
        "*", "*"]
    ]
  }
}
```

A-2. 固有表現抽出

リクエスト URL

```
https://jlp.yahooapis.jp/NERService/V1/extract
```

リクエストパラメータ

パラメータ	必須	値	説明
id	○	string, integer	例: 1-234-D
jsonrpc	○	string	2.0 (固定)
method	○	string	jlp.nerservice.extract (固定)
params	○	object	
params/q	○	string	解析対象テキスト

サンプルリクエスト

```
{
  "id": "1234-1",
  "jsonrpc": "2.0",
  "method": "jlp.nerservice.extract",
  "params": {
    "q": "加藤さんは10時に東京駅に到着しました"
  }
}
```

レスポンスフィールド

フィールド	値	説明
id	string, integer	リクエストの id の値が返ります
jsonrpc	string	固定で「2.0」が返ります
result	object	固有表現の抽出結果
result/ entityList	array (array)	固有表現の抽出結果を含む配列 抽出結果が存在しない場合は空です
result/ entityList/ text	string	固有表現のテキスト
result/ entityList/ type	string	固有表現の種類 以下 8 つのいずれかを返します - ORGANIZATION: 組織名 - PERSON: 人名 - LOCATION: 地名 - ARTIFACT: 固有物名 - DATE: 日付表現 - TIME: 時間表現 - MONEY: 金額表現 - PERCENT: 割合表現
result/ entityList/ offset	integer	固有表現の文字オフセット
result/ entityList/ length	integer	固有表現の文字長

サンプルレスポンス

```
{
  "id": "1234-1",
  "jsonrpc": "2.0",
  "result": {
    "entityList": [
      {
        "length": 2, "offset": 0,
        "text": "加藤",
        "type": "PERSON"
      },
      {
        "length": 3, "offset": 5,
        "text": "10時",
        "type": "TIME"
      },
      {
        "length": 3, "offset": 9,
        "text": "東京駅",
        "type": "LOCATION"
      }
    ]
  }
}
```

A-3. キーフレーズ抽出

リクエスト URL

https://jlp.yahooapis.jp/KeyphraseService/V2/extract

リクエストパラメータ

パラメータ	必須	値	説明
id	○	string, integer	例: 1-234-D
jsonrpc	○	string	2.0 (固定)
method	○	string	jlp.keyphraseservice. extract (固定)
params	○	object	
params/q	○	string	解析対象テキスト

サンプルリクエスト

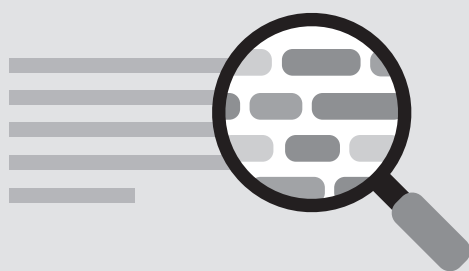
```
{
  "id": "1234-1",
  "jsonrpc": "2.0",
  "method": "jlp.keyphraseservice.extract",
  "params": {
    "q": "東京ミッドタウンから国立新美術館まで歩いて
          5分で着きます。"
  }
}
```

レスポンスフィールド

フィールド	値	説明
id	string, integer	リクエストの id の値が返ります
jsonrpc	string	固定で「2.0」が返ります
result	object	キーフレーズ抽出結果
result/ phrases	array (object)	キーフレーズ抽出結果のリスト (最大 20 個)
result/ phrases/ score	integer	キーフレーズの重要度 最大値を 100 とし、以下相 対的な重要度に応じて 100 以下の値が割り当てられま す
result/ phrases/ text	string	キーフレーズ

サンプルレスポンス

```
{
  "id": "1234-1",
  "jsonrpc": "2.0",
  "result": {
    "phrases": [
      {
        "score": 100,
        "text": "東京ミッドタウン"
      },
      {
        "score": 73,
        "text": "国立新美術館"
      },
      {
        "score": 37,
        "text": "5分"
      }
    ]
  }
}
```



テキスト解析 Web API ガイドブック

手を動かして学ぶ 日本語テキスト処理

発行日	2026年3月1日
著者	LINEヤフー株式会社 テキスト解析ガイドブック制作チーム 井上 裁都・岩間 寛悟・西山 育宏・牧野 恵・山下 達雄 ※所属は、執筆当時（2025年10月）のもので
デザイン	岡田 紗代子
発行	LINEヤフー株式会社 〒102-8282 東京都千代田区紀尾井町 1-3 東京ガーデンテラス紀尾井町 紀尾井タワー
印刷所	キンコーズ・ジャパン株式会社